BASH INTRO

CONTENT

I/O streams and Redirection:

- stdin
- stdout
- stderr

Exit Status

Control Operators

Regex

File Manipulation commands :

- sort
- uniq
- cut
- paste
- comm
- tr
- sed
- awk

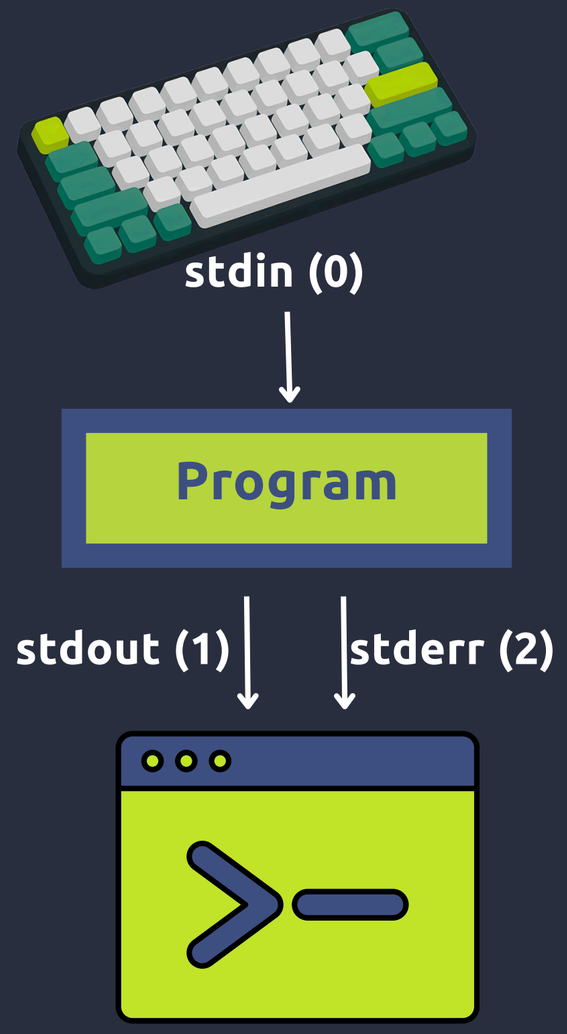
I/O Streams

Every Unix process has three standard data streams automatically connected to it.

stdin (standard input): Data fed into a program
Default source: keyboard
Numeric Descriptor: 0
Redirect with **<** or pipe **|**

stdout (standard output): Normal results printed by a program
Default Destination : Terminal Screen
Numeric Descriptor: 1
Redirect with **>**

stderr (standard error): Error messages and warnings
Separate from stdout so errors don't corrupt your data output.
Default Destination: Terminal Screen
Numeric Descriptor: 2
Redirect with **2>**



I/O Streams Redirection (1)

> file : Redirects stdout to file :

```
grep “TATA” sequences.fasta >TATA_seq.fasta
```

2> file : Redirects stderr to file :

```
grep “TATA” sequences.fasta 2> error.fasta
```

>&2 : Redirects stdout to stderr :

```
echo “Process has started...”  >&2
```

2>&1 : Redirects stderr to stdout :

```
bwa mem ref.fa reads.fq > aln.log 2>&1
```

&> file : Redirects both stdout and stderr to file :

```
fastqc sample.fastq &> fastqc.log
```

I/O Streams Redirection (2)

In bioinformatics, tools like BLAST, BWA, STAR, and samtools send their progress logs to stderr and actual data to stdout . Always separate them so your output files stay clean.

Separating good results from errors in a BLAST search :

```
blastn -query sequences.fasta -db nt -out results.txt 2>blast_errors.log
```

results.txt has your hits, blast_errors.log has any warnings/errors.
You can inspect them independently.

I/O Streams Redirection (3)

Logging pipeline progress with >&2 :

```
#!/bin/bash
# align_and_call.sh — BWA alignment + variant calling

echo "Starting alignment..." >&2
bwa mem genome.fa reads.fastq > aligned.sam

echo "Calling variants..." >&2
samtools sort aligned.sam | bcftools call > variants.vcf

echo "Done! Variants saved to variants.vcf" >&2
```

Using >&2 for status messages is best practice : it keeps your actual data (stdout) clean and pipeable, while progress info goes to stderr.

I/O Streams Redirection (4)

! Order matter !

✅ correct order: stderr goes to results.txt

```
blastn -query seq.fa -db nt > results.txt 2>&1
```

❌ wrong order: stderr goes to terminal, not to the file

```
blastn -query seq.fa -db nt 2>&1 > results.txt
```

Discarding Streams (/dev/null)

If a tool is too verbose and you don't care about the logs or the errors, you can redirect the stream to /dev/null, which acts as a "black hole."

Silence all error messages and logs, keeping only the stdout :

```
grep "missense_variant" annotations.vcf 2> /dev/null
```

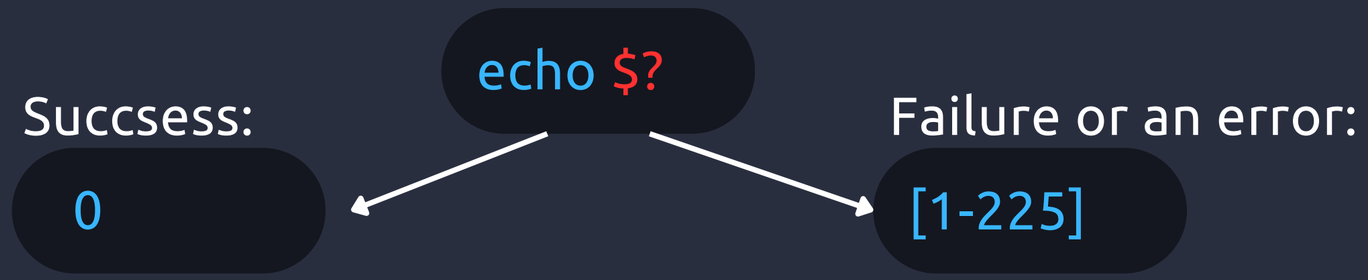
Some tools like samtools print stats to stderr you don't always need :

```
samtools view -bS aligned.sam 2>/dev/null > aligned.bam
```

Exit Status

$?

It stores the exit status of the last command executed in the foreground.




```
bwa mem genome.fa reads.fastq > aligned.sam 2> bwa.log
if [ $? -ne 0 ]; then
    echo "ERROR: Alignment failed!" >&2
    exit 1
fi
echo "Alignment is successful, process conprocess continuing..." >&2
```

Control Operators

&& (AND) Operator

Runs the next command only if the previous one succeeded (exit status = 0)

```
fastqc sample.fastq && echo "QC completed"
```

|| (OR) Operator

Runs the next command only if the previous one failed (exit status ≠ 0):

```
fastqc sample.fastq || echo "QC error"
```

; Operator

Runs second command NO MATTER WHAT

```
fastqc sample.fastq ; echo "done"
```

Escape Sequences

Escape sequences use a backslash (\) to tell the computer to treat the following letter as a special command rather than literal text.

\n It tells the cursor to jump down to the start of the next line.

\t It inserts a "tab" space, which is usually equal to 4 or 8 regular spaces.

```
H.sapiens\nM.musculus\nD.rerio
H.sapiens
M.musculus
D.rerio
```

```
H.sapiens\tM.musculus\tD.rerio
H.sapiens    M.musculus    D.rerio
```

TSV: Tab-Separated Values
delimiter : **\t**

CSV: Comma-Separated Values
delimiter : **,**

Bash Regex

A regular expression (regex) is a specialized language used to search for patterns within text. Instead of looking for a fixed word, you define a rule (pattern), and any text that matches that rule is returned.

grep flags for Regex:

-P (PCRE mode) : It uses Perl regex engine.
-E (Extended Regex / ERE): It is a POSIX standard and does not recognize shortcuts such as \d, \w, and \s.

\t tab
\n newline
\d Digit [0-9]
\D Non-digit
\w Word char [a-zA-Z0-9_]
\W Non-word char
\s Whitespace
\S Non-whitespace

Regex (1)

dot (.) : Matches any one character except a newline.
Useful for fuzzy matching in DNA sequences where one base may be unknown.

```
echo "ATGCATGC" | grep -oP "A.G"
```

Matches: ATG , ACG

caret (^) : Anchors the matches to the beginning of a line.

```
grep "^>" sequences.fasta
```

Matches all FASTA header lines .

dollar sign ($) : Anchors the match to the end of a line.

```
grep "TAA$" sequences.txt
```

Matches sequences ending with stop codon TAA .

Regex (2)

asterix (*) : Matches zero or more of the preceding character.

```
grep -E "GT*A" sequences.txt
```

Useful for finding variable-lenfth repeat regions.

plus (+) : Matches one or more of the preceding character/group.

```
grep -E "A+" sequences.txt
```

Matches one or more consecutive A's (poly-A tails!)

question mark (?) : Makes the pereceding character optional.

```
echo "color , colour" | grep -E "colou?r"
```

Matches: color and colour

```
grep -E "ATG(C)?AT" sequences.txt
```

Matches: ATGAT and ATGCAT

Regex (3)

pipe (|) : Matches either the left or right pattern.

```
grep -E "TAA|TAG|TGA" sequences.txt
```

Matches any of the 3 stop codons.

```
grep -E "Homo sapiens|Mus musculus|Dania rerio" organisms.txt
```

Filter for 3 model organisms.

parentheses ( ) : Groups patterns together and captures them for backreferences.

```
grep -oP "(ATG)+" sequences.txt
```

Matches repeated ATG motifs : ATG, ATGATG, ATGATGATG

```
grep -E "(GT){2,}" introns.txt
```

Matches GT repeated 2 or more times (splice site patterns)

Regex (4)

Quantifier ({}): Specifies exact or min-max repetitions.

```
grep -E "A{3,6}" sequences.txt
```
Matches runs of 3 to 6 A's : AAA, AAAA, AAAAA, AAAAAA

```
grep -E "[CG]{4}" sequences.txt
```
Matches exactly 4 consecutive C or G bases (GC-rich regions)

```
grep -E "(CAG){10,35}" huntingtin.txt
```
CAG repeat expansion : 10–35 is normal range in HTT gene

parentheses (\ ): Removes special meaning from the next character, or creates special sequences.

```
grep "3\.14" lab_values.txt
```
Matches literal "3.14", not "3X14"

```
grep -P "\d+"  read_counts.txt
```
\d = any digit : matches read counts

```
grep -P "\t" tsv_file.tsv
```
\t = tab

Regex (5)

Character Class ([ ]) : Matches any one character inside the brackets.

```
grep -E “[ACGT]+” sequences.txt
```

Matches valid DNA bases only

```
grep -E “[^ACGT]” sequences.txt
```

^ inside [] means NOT :
finds invalid/ambiguous bases like N, R, Y

```
grep -E "[0-9]+" quality_scores.txt
```

Matches any number in quality score data

```
grep -E "chr[0-9XY]+" genome.bed
```

Matches chromosome names: chr1, chr2 ... chrX, chrY

Regex (6)

```
grep -P "^chr\w+\t\d+\t\d+" genome.bed
```

Matches BED format:
chrName <tab> start <tab> end

The BED (Browser Extensible Data) format is a text file used in bioinformatics to store genomic coordinates and associated annotations.
A BED file is tab-delimited and consists of 3 to 12 columns. The first three columns are mandatory, while the remaining nine are optional.

1	chr7	127471196	127472363
2	chr7	127472363	127473530
3	chr7	127473530	127474697
4	chr7	127474697	127475864
5	chr7	127475864	127477031
6	chr7	127477031	127478198
7	chr7	127478198	127479365
8	chr7	127479365	127480532
9	chr7	127480532	127481699

chrName <tab> **start** <tab> **end**

Regex (7)

Find microsatellite repeats (e.g. AC repeated 5+ times):

```
grep -P "(AC){5,}" genome.txt
```

Extract all FASTA headers containing human proteins :

```
grep -E "^>.*[Hh]omo sapiens" proteome.fasta
```

Find all start-codon-to-stop-codon patterns (simple ORF finder):

```
grep -oP "ATG[ACGT]{3,}?(TAA|TAG|TGA)" sequence.txt
```

sort

sort reorders lines of text.
By default it sorts alphabetically.
In bioinformatics you sort gene expression tables, VCF files, BED coordinates, and more.

```
sort gene_ids.txt
```

-n numeric sort (for numbers)	`sort -n gene_ids.txt`
-r reverse order	`sort -r gene_ids.txt`
-k sort by column (1-indexed)	`sort -k3 gene_ids.txt`
-t field separator (-t$'\t' , -t,)	`sort -t$'\t' -k4 gene_ids.tsv`
-u output unique lines only	`sort -u gene_ids.txt`
-V version sort (chr1 chr2 chr10 order!)	`sort -V file.txt`

uniq

uniq command collapses consecutive identical lines.
uniq only removes (or reports) adjacent duplicate lines. If your duplicates aren't next to each other, uniq won't consider them duplicates and they will not be removed/countable as duplicates.
Critical for counting unique transcripts, genes, samples, etc.

```
sort gene_ids.txt | uniq
```

```
uniq gene_ids.txt
```

-c Prefix lines with count

```
sort gene_ids.txt | uniq -c
```

-d Print only duplicate lines

```
sort gene_ids.txt | uniq -d
```

-u Print only unique lines (appearing once)

```
sort gene_ids.txt | uniq -u
```

-i Case-insensitive

```
sort gene_ids.txt | uniq -i
```

sort & uniq examples (1)

```
species.txt:
Homo sapiens
Mus musculus
Homo sapiens
Drosophila melanogaster
Mus musculus
```

Count how many times each species appears → Then sort by frequency

```
sort species.txt | uniq -c
# Output:
#   1 Drosophila melanogaster
#   2 Homo sapiens
#   3 Mus musculus
```

```
sort species.txt | uniq -c | sort -rn
#Output:
#   3 Mus musculus
#   2 Homo sapiens
#   1 Drosophila melanogaster
```

sort & uniq examples (2)

Find duplicate gene IDs in an annotation file :

```
sort genes.txt | uniq -d > duplicate_genes.txt
```

genes.txt has one gene ID per line extracted from a GFF/GTF

```
sort genes.txt | uniq -u > unique_genes.txt
```

Print only genes that appear exactly once (no duplicates)

Count unique variants per chromosome from a VCF :

```
grep -v "^#" variants.vcf | cut -f1 | sort | uniq -c | sort -rn

# Output:
#  4521 chr1
#  3812 chr2
#  2900 chr3
#   ...
```

Extract chromosome column from a VCF (skip header lines)

cut (1)

The cut command is used to extract specific sections (columns, characters, or bytes) from each line of a file.

Field =Column

GeneID	Chr	Strand	Pathway
BRCA1	chr17	-	DNA_repair
BRCA2	chr13	+	DNA_repair
TP53	chr17	+	Tumor_Suppressor
MYC	chr8	-	Oncogene

```
cut -f1 samples.tsv
```

Chr column

```
cut -f1,3 samples.tsv
```

GeneID column
Strand column

```
cut -f1-3 samples.tsv
```

GeneID column
Chr column
Strand column

cut (2)

-d (delimiter): Defines the character that separates your data (e.g., a comma, space, or colon).
-c (characters): Extracts specific character positions (e.g., the first 5 letters).
--complement : TakeS everything except what you specified.

```
cut -d',' -f1,3 samples.csv
```

comma-separated

```
cut -d':' -f1 /etc/passwd
```

colon-separated (usernames!)

```
echo "ATGCATGCATGC" | cut -c1-3
```

ATG (first 3 chars)

```
cut -f2 --complement samples.tsv
```

Returns all columns EXCEPT column 2

paste (1)

The paste command merges files side by side, joining lines column by column.

GeneID
AKT1
MTOR
JAK2
STAT3

Chr
chr14
chr1
chr9
chr17

```
paste geneID.txt Chr.txt > unique_genes.txt
```

GeneID	Chr
AKT1	chr14
MTOR	chr1
JAK2	chr9
STAT3	chr7

paste (2)

-d (delimiter): Sets the character used to join the columns (the default is Tab).
-d‘ ’ : Use custom delimiter between joined columns

```
paste -d',' GeneID.txt Chr.txt
```

AKT1,chr14

```
paste -d'\t' GeneID.txt Chr.txt
```

AKT1 chr14

```
paste -d'|' GeneID.txt Chr.txt
```

AKT1|chr14

-s (serial): Merges all lines of a single file into one long horizontal line.

```
paste -s -d',' GeneID.txt
```

AKT1,MTOR,JAK2,STAT3

comm (1)

comm compares two sorted files line by line and shows what's unique to each file and what's shared.

```
comm file1.txt file2.txt
```

col 1	col 2	col 3
Only in file1	Only in file2	In Both file

comm (2)

Find shared genes between two RNA-seq experiments

```
comm -12 <(sort rna_exp1.txt) <(sort rna_exp2.txt)
```

Find variants ONLY in tumor, not in normal sample

```
comm -23 <(sort tumor_variants.txt) <(sort normal_variants.txt)
```

Find novel variants (not in known database)

```
comm -23 <(sort my_variants.txt) <(sort dbSNP.txt) > novel_variants.txt
```

tr (1)

tr command translates, deletes, or squeezes characters from standard input and write the result to standard output.

It works character by character and only reads from stdin !
You must | or use <

`tr 'T' 'U' < rna.txt`	DNA to RNA
`echo "ATGC" | tr 'ATGC' 'TACG'`	Reverse complement
`echo "atgcatgc" | tr 'a-z' 'A-Z'`	Lowercase to uppercase
`echo "ATG CAT GCA" | tr -d ' '`	Remove all spaces
`tr -d '\n' < sequence.fasta`	Remove newlines (make multi-line sequence one line)

tr (2)

Delete everyting except ACGT and newline

```
echo “ATG-CAT_123_GCA” | tr -cd ‘ACGT\n”
```

Clean up Windows line endings (\r\n → \n)

```
tr -d '\r' < windows_file.txt > unix_file.txt
```

Collapse multi-line FASTA sequence to single line

```
grep -v '^>' sequence.fasta | tr -d '\n'
```

sed-Stream Editor

sed command finds, replaces, deletes, inserts text.

Substitution : s/find/replace/

Converts DNA to RNA (T → U), g = global (all occurrences) :

```
sed 's/T/U/g' dna.txt
```

Abbreviate species names throughout a file :

```
sed 's/Homo sapiens/H.sapiens/g' species.txt
```

Convert to uppercase :

```
sed 's/[a-z]/\U&/g' genes.txt
```

Replace only the 2nd ATG :

```
sed 's/ATG/[START]/2' seq.txt
```

sed-Stream Editor (2)

Print Specific Lines : p with -n

```
sed -n '1p' file.fasta
```

Print only line 1

```
sed -n '1,4p' file.fasta
```

Print lines 1 to 4

```
sed -n '/^>/p' file.fasta
```

Print only FASTA header lines

-n suppresses default printing , p then explicitly prints matches.

Delete Lines : d

```
sed '/^#/d' annotation.gff
```

Remove comment lines starting with #

```
sed '/^$/d' sequences.txt
```

Remove blank lines

```
sed '/^>/d' file.fasta
```

Remove FASTA headers, keep sequences only

sed-Stream Editor (3)

In-Place Editing : -i

```
sed -i 's/chr/Chr/g' genome.bed
```
Edit file directly

```
sed -i.bak 's/T/U/g' dna.txt
```
Edit file, keep backup as dna.txt.bak

Insert and Append : i and a

```
sed '1i\##Pipeline output v1.0' results.txt
```
Insert line BEFORE line 1

```
sed '$a\##End of file' results.txt
```
Append line AFTER last line

```
sed '/^>/a\ORIGIN'  file.fasta
```
Append after every header

sed-Stream Editor (4)

Adress Ranges

`sed '5,10d' file.txt`	Delete lines 5 to 10
`sed '/START/,/END/d' file.txt`	Delete everything between START and END

Examples:

`sed 's/^/chr/' genome.bed`	Rename chromosomes from “1” to “chr1” in a BED file
`sed '/^>/d' proteome.fasta`	Extract only sequences from a FASTA (remove headers)
`sed 's/^/GENE_/' gene_ids.txt`	Add “GENE_” prefix to every gene ID

awk

Alfred Aho
Peter Weinberger
Brian Kernighan

awk is a mini-language for processing structured text.
It processes line by line and splits each line into fields ($1, $2...) separeted by whitespace (or a custom delimiter).

$0 = entire current line
$1 = first field
$2 = second field
NR = Current line number
NF = Number of fields in current line
FS = Field separator (default :whitespace)
OFS = Output field separator

$1	$2	$3	$4
GeneID	**Chr**	**Strand**	**Pathway**
BRCA1	chr17	-	DNA_repair
BRCA2	chr13	+	DNA_repair
TP53	chr17	+	Tumor_Suppressor
MYC	chr8	-	Oncogene

$0

awk (2)

Print Specific Columns

```
awk '{print $1}' genome.bed
```

Print chromosome column

```
awk '{print $NF}' file.txt
```

Print last column

```
awk '{print $1, $2, $3 }' genome.bed
```

Print first 3 columns (chr, start, end)

awk (3)

Filtering with Conditions

`awk '$5 > 100 ' expression.tsv`	Keep genes with expression > 100
`awk '$3 - $2 > 1000' genes.bed`	Keep genes longer than 1000bp
`awk 'NR > 1' results.tsv`	skip header line (line1)
`awk '$1 == "chr1"' genome.bed`	Only chromosome 1
`awk '$6 == "+"' genes.bed`	Only forward strand genes

awk (4)

Custom Field Separators : F

```
awk -F'\t' '{print $1, $4}' file.tsv
```

Tab-separated file

```
awk -F',' '{print $2}' file.csv
```

CSV file

```
awk -F':' '{print $1}' /etc/passwd
```

Colon-separeted

VCF files are tab-separated

```
awk -F'\t' '$1 != "#" {print $1, $2, $4, $5}' variants.vcf
```

Prints: CHROM POS REF ALT

awk (5)

Reformating Files

```
awk ‘{print $1":“$2”-”$3"\t”$4}’ genes.bed
```

Covert BED to custom format

```
awk ‘{print $2, $1}’ file.txt
```

Swap columns 1 and 2

```
awk ‘OFS=”\t” {print $0, $3-$2}’ genes.bed
```

Add a new calculated column (gene lenght)

```
awk -F’,’ ‘BEGIN{OFS}”\T} {$1=$1; print}’ file.csv
```

Covert CSV to TSV

awk (6)

Pattern Matching in awk

`awk '/>/' file.fasta`	Print FASTA header lines
`awk '!/>/' file.fasta`	Print sequence lines only
`awk '/TATA/' promoters.txt`	Lines containing TATA box
`awk '$4 ~ /BRCA/' annotation.gtf`	Columns 4 contain “BRCA”
`awk '$4 !~ /pesudo/' annotation.gtf`	Columns 4 does NOT contain “pseudo”

awk (7)

Calculating Statistics

Sum all values in column 2

```
awk '{sum += $2} END {print "Total:", sum}' counts.txt
```

Calculate average expression

```
awk 'NR>1 {sum += $2; count++} END {print "Mean:", sum/count}' expression.txt
```

Find min and max read length

```
awk 'NR%4==2 {len=length($0); if(len>max)max=len; if(len<min||NR==2)min=len} \
END {print "Min:", min, "Max:", max}' reads.fastq
```

awk (8)

Bioinformatics Examples

Get the top 5 most expressed genes

```
awk ‘NR>1 {print $2, $1}’ expression.txt | sort -rn | head -5
```

Count sequences in a FASTA file

```
awk '/^>/{count++} END{print count}' proteome.fasta
```

Calculate GC content per sequence line

```
awk '!/^>/ {
        gc = gsub(/[GC]/, "&")
        total = length($0)
        print gc/total * 100 "% GC"
}' sequences.fasta
```

Sources

- https://astrobiomike.github.io/unix/six-glorious-commands
- Future Learn-Bioinformatics for Biologists
- https://rsh249.github.io/bioinformatics/unix_shell.html#searching_and_manipulating_text_files:_grep,_awk,_shuf,_sort,_uniq
- https://www.freecodecamp.org/news/the-linux-awk-command-linux-and-unix-usage-syntax-examples/
- https://www.youtube.com/watch?v=sAD4dq_jDTA&list=PLShDm2AZYnK1SdG3dufPdCqk08sOahUBP